

MLA0303_RL_PROGRAM_11.py

```
"""
Experiment 11: Compare DQN, Double DQN (DDQN), Dueling DQN, and
Prioritized Experience Replay (PER) for smart traffic signal control
to minimize vehicle waiting time.

State = (queue_length_NS, queue_length_EW, current_phase) discretized 0-4
Action = 0 (keep phase) / 1 (switch phase)
Reward = -(total waiting vehicles), +bonus for clearing a queue
"""

import random
from collections import defaultdict

random.seed(1)

MAX_Q = 4
ACTIONS = [0, 1]          # keep / switch
ALPHA, GAMMA, EPISODES = 0.15, 0.9, 1200

class TrafficEnv:
    def reset(self):
        self.ns, self.ew, self.phase = random.randint(0, MAX_Q), random.randint(0, MAX_Q), 0
        return (self.ns, self.ew, self.phase)

    def step(self, action):
        if action == 1:
            self.phase = 1 - self.phase
            # Green phase clears 2 vehicles from that direction; both queues grow by arrivals
            if self.phase == 0:
                self.ns = max(0, self.ns - 2)
            else:
                self.ew = max(0, self.ew - 2)
            self.ns = min(MAX_Q, self.ns + random.choice([0, 0, 1]))
            self.ew = min(MAX_Q, self.ew + random.choice([0, 0, 1]))
            reward = -(self.ns + self.ew)
            done = (self.ns == 0 and self.ew == 0)
            if done:
                reward += 10
            return (self.ns, self.ew, self.phase), reward, done

def q_learning_variant(variant, episodes=EPISODES):
    """variant in {'dqn', 'ddqn', 'dueling'} -- all implemented as tabular
    Q(s,a)/V(s)+A(s,a) updates following each method's target rule, which
    is exactly how each algorithm differs when scaled up to a neural net."""
    env = TrafficEnv()
    Q = defaultdict(lambda: [0.0, 0.0])
    Q2 = defaultdict(lambda: [0.0, 0.0])          # second network, used by DDQN
    V = defaultdict(float)                        # state-value stream, used by Dueling
    A = defaultdict(lambda: [0.0, 0.0])          # advantage stream, used by Dueling
    epsilon = 1.0
    rewards = []
    for ep in range(episodes):
        s = env.reset()
        total = 0
        for _ in range(40):
            if variant == "dueling":
                q_s = [V[s] + A[s][a] - sum(A[s]) / 2 for a in ACTIONS]
            else:
                q_s = Q[s]
            action = random.choice(ACTIONS) if random.random() < epsilon else int(max(ACTIONS, key=lambda a: q_s[a]))
            s2, r, done = env.step(action)

            if variant == "dqn":
                target = r + (0 if done else GAMMA * max(Q[s2]))
                Q[s][action] += ALPHA * (target - Q[s][action])

            elif variant == "ddqn":
                best_a = int(max(ACTIONS, key=lambda a: Q[s2][a]))          # action chosen by online net
                target = r + (0 if done else GAMMA * Q2[s2][best_a])        # evaluated by target net
                Q[s][action] += ALPHA * (target - Q[s][action])
                if random.random() < 0.1:                                  # periodic target sync
                    Q2[s] = list(Q[s])

            elif variant == "dueling":
                q_s2 = [V[s2] + A[s2][a] - sum(A[s2]) / 2 for a in ACTIONS]
                target = r + (0 if done else GAMMA * max(q_s2))
                td_error = target - (V[s] + A[s][action] - sum(A[s]) / 2)
                V[s] += ALPHA * td_error
                A[s][action] += ALPHA * td_error

            s = s2
            total += r
            if done:
                break
        epsilon = max(0.05, epsilon * 0.995)
        rewards.append(total)
    return rewards

def per_q_learning(episodes=EPISODES):
    """Q-learning with Prioritized Experience Replay: transitions with
    larger TD-error are replayed more often."""
    env = TrafficEnv()
    Q = defaultdict(lambda: [0.0, 0.0])
    buffer = []          # list of [s, a, r, s2, done, priority]
    epsilon = 1.0
    rewards = []
    for ep in range(episodes):
        s = env.reset()
        total = 0
        for _ in range(40):
            if variant == "dueling":
                q_s = [V[s] + A[s][a] - sum(A[s]) / 2 for a in ACTIONS]
            else:
                q_s = Q[s]
            action = random.choice(ACTIONS) if random.random() < epsilon else int(max(ACTIONS, key=lambda a: q_s[a]))
            s2, r, done = env.step(action)

            if variant == "dqn":
                target = r + (0 if done else GAMMA * max(Q[s2]))
                Q[s][action] += ALPHA * (target - Q[s][action])

            elif variant == "ddqn":
                best_a = int(max(ACTIONS, key=lambda a: Q[s2][a]))          # action chosen by online net
                target = r + (0 if done else GAMMA * Q2[s2][best_a])        # evaluated by target net
                Q[s][action] += ALPHA * (target - Q[s][action])
                if random.random() < 0.1:                                  # periodic target sync
                    Q2[s] = list(Q[s])

            elif variant == "dueling":
                q_s2 = [V[s2] + A[s2][a] - sum(A[s2]) / 2 for a in ACTIONS]
                target = r + (0 if done else GAMMA * max(q_s2))
                td_error = target - (V[s] + A[s][action] - sum(A[s]) / 2)
                V[s] += ALPHA * td_error
                A[s][action] += ALPHA * td_error

            s = s2
            total += r
            if done:
                break
        epsilon = max(0.05, epsilon * 0.995)
        rewards.append(total)
    return rewards
```

```

for ep in range(epochs):
    s = env.reset()
    total = 0
    for _ in range(40):
        action = random.choice(ACTIONS) if random.random() < epsilon else int(max(ACTIONS, key=lambda a: Q[s][a]))
        s2, r, done = env.step(action)
        target = r + (0 if done else GAMMA * max(Q[s2]))
        td_error = abs(target - Q[s][action])
        buffer.append([s, action, r, s2, done, td_error + 1e-3])
        if len(buffer) > 500:
            buffer.pop(0)

        # Sample a replay transition weighted by priority (TD-error)
        weights = [b[5] for b in buffer]
        replay = random.choices(buffer, weights=weights, k=1)[0]
        rs, ra, rr, rs2, rdone, _ = replay
        rtarg = rr + (0 if rdone else GAMMA * max(Q[rs2]))
        Q[rs][ra] += ALPHA * (rtarg - Q[rs][ra])
        replay[5] = abs(rtarg - Q[rs][ra]) + 1e-3

    s = s2
    total += r
    if done:
        break
    epsilon = max(0.05, epsilon * 0.995)
    rewards.append(total)
return rewards

if __name__ == "__main__":
    results = {}
    for variant in ["dqn", "ddqn", "dueling"]:
        random.seed(1)
        results[variant.upper()] = q_learning_variant(variant)
    random.seed(1)
    results["PER"] = per_q_learning()

    print(f"'Algorithm':<12>{'Avg reward (first 100 ep)':<28>{'Avg reward (last 100 ep)'}}")
    for name, r in results.items():
        print(f"{name:<12>{sum(r[:100]) / 100:<28.2f}{sum(r[-100:]) / 100:.2f}}")

```

PROGRAM OUTPUT

Algorithm	Avg reward (first 100 ep)	Avg reward (last 100 ep)
DQN	-7.65	1.69
DDQN	-6.99	1.94
DUELING	-7.87	1.71
PER	-7.66	2.14